Reserve-a-Lab load and concurrency report

Performance efficiency testing for Objective 2: load, stress, spike, soak and double-booking tests with Grafana k6 against local XAMPP on 27 October 2026, plus paired statistics for the Lighthouse before/after results. Companion to the Lighthouse performance report.

Capacity (stress test)	p95 at expected peak (50 users)	Worst p95, stress test	Double booking, before fixes
≥ 200 virtual users	187 ms	4.14 s at 200 users	FAIL 20 of 20 approved

Double booking, after fixes	Lighthouse score, paired t-test
PASS max 1 of 20 approved	+25.1 p < .001

Main findings

- **Time behaviour is good up to 100 concurrent users.** At the expected peak of 50 users the median response was 25 ms and the p95 187 ms, with no errors in 17,182 requests. Up to 100 users the p95 stayed below 200 ms.
- **Capacity is at least 200 users** (p95 ≤ 10 s and errors ≤ 5%). At 200 users the median stayed at 24 ms but the p95 rose to 4.14 s: about one request in ten waited 2–6 s. The 300-user step was stopped by the host for low memory before it finished, so the upper limit was not measured.
- **The 200-user slowdown is most likely Apache's thread limit, not CPU.** Apache on Windows serves each connection with one of 150 threads and keeps idle connections open for 5 s. With more users than threads, a request on a new connection waits for a thread; the delay sits in time-to-first-byte, is bounded by the 5 s keep-alive timeout, and appears only above 150 users, while CPU averaged 34% of the machine.
- **Response time climbs as data accumulates.** In the 30-minute soak, the admin approvals page went from a median of 166 ms in the first 5 minutes to 4.9 s by minute 25, because it renders every pending request on one page (about 4,000 by the end). Apache and MariaDB CPU rose with it at constant load. Memory showed no growth in the first 15 minutes, before other programmes on the laptop began to interfere.

- **Double booking: FAIL before the fixes** (PASS after them: 1 approved of 20 in every repetition). When 20 approvals of the same lab, date and time were sent at once, 20, 20, 19, 18, 17 of 20 were approved in the five repetitions (the correct number is 1). It also happens without any concurrency: approved 3 seconds apart through the real page, all 3 of 3 were approved.
- **Inventory over-allocation: FAIL before the fixes** (PASS after them: 2 approved, stock never below zero). 20 requests for 2 of the last 5 units were all approved in every repetition (40 units allocated against 5 in stock), and the stock went below zero (−1) in 2 of 5 repetitions. The inventory page's manual deduction, which locks the row, passed the same test 5 of 5 times.
- **The Lighthouse improvements are statistically significant** for performance score (+25.1 points, paired t-test $p < .001$), FCP (−2.22 s, t-test $p < .001$), LCP (−5.00 s, Wilcoxon p .0078) and page weight (−1,246 KB, Wilcoxon p .0078), n = 8 pages.

Test setup

Item	Value
Machine	Laptop, Intel Core i5-1240P (12 cores / 16 threads), 15.7 GB RAM, Windows 11 Home 10.0.26200
Web stack	XAMPP 8.2.12: Apache 2.4.58 (mpm_winnt, ThreadsPerChild 150, KeepAlive On, KeepAliveTimeout 5 s), PHP 8.2.12 as Apache module, OPcache off
Database	MariaDB 12.3.2 (system tables still from 10.4.8), InnoDB, REPEATABLE-READ, autocommit on, max_connections 151, innodb_buffer_pool_size 16 MB
Load tool	Grafana k6 v2.2.0 (go1.26.5), on the same machine as the server
Resource sampling	Windows performance counters (Get-Counter) every 2 s: httpd.exe and mysqld.exe CPU and private working set, k6 CPU, total CPU, available memory
Date	27 October 2026
Target	http://localhost/reserve-a-lab/ only (scripts refuse any other host)

User journeys

Each virtual user repeats a complete session, from the login page to logging out, with think times between steps. 80% of users follow the requester journey and 20% the admin journey. Every session logs in as its own synthetic account (500 teachers, 5 admins).

Requester (teacher)

1. Open the login page (`index.php`) and read the CSRF token
2. Log in (`ajax_login.php`)
3. Think 2–4 s
4. Requester home (`requester_home.php`) with its two on-load calls
5. Think 3–6 s
6. Open the request form (`forms.php`) with its lookups (disabled dates, subjects, sections) and the live conflict check
7. Think 5–10 s (filling in the form)
8. Conflict check at the confirmation step
9. Think 2–4 s
10. Submit the request with 1–2 materials (`ajax_supervisor_action.php`)
11. Think 2–4 s
12. Request tracker (`requests.php`)
13. Think 2–4 s
14. Log out

Admin (lab personnel)

1. Open the login page and log in
2. Think 2–4 s
3. Admin home (`admin_home.php`) with its three on-load calls
4. Think 3–6 s
5. Admin approvals (`admin_approve.php`), pick a pending load-test request, conflict check (as the page sends it)
6. Think 3–6 s
7. Approve it (`ajax_admin_action.php` , force_approve)
8. Think 2–4 s
9. Inventory (`inventory.php`)
10. Think 2–4 s
11. Log out

k6 requests the HTML pages and the AJAX calls the pages make; it does not download CSS, scripts or images (front-end loading is covered by the Lighthouse report). Two conflict-check calls are replayed exactly as the pages send them and return HTTP 403 (see the double-booking section); these are expected responses, not errors.

Safety and test data

- Only localhost was tested. The database `dbadmin` was backed up with mysqldump before testing and restored afterwards; all 47 tables matched their pre-test checksums after the restore.
- Every test started from the same baseline (original data plus synthetic accounts and test items), restored before each run, so later tests are not affected by rows created by earlier ones.
- Submissions and approvals send notification e-mails through Gmail SMTP inside the request. Outbound SMTP from Apache was blocked with a Windows Firewall rule for the duration (connections fail in under 40 ms), and every address in the database was replaced with an `@example.invalid` address. No e-mail left the machine.
- Load-journey materials used 10 synthetic consumables with 1,000,000 units each, so real stock was never drawn down.
- No application code was changed. The inventory deduction log table, which the inventory page needs but which was not present locally (its migration was removed in commit 8f9d971), was created from that migration for the control test and removed by the restore.

Definitions

Response time = k6 `http_req_duration` (request sent to last byte received). Percentiles use linear interpolation. Error rate = requests with an HTTP error, timeout, connection error or functional failure (for example a lost session or a refused submission) divided by all requests. CPU is shown as a percentage of the whole machine (16 logical processors), so 6.25% equals one fully busy core. Memory is the private working set.

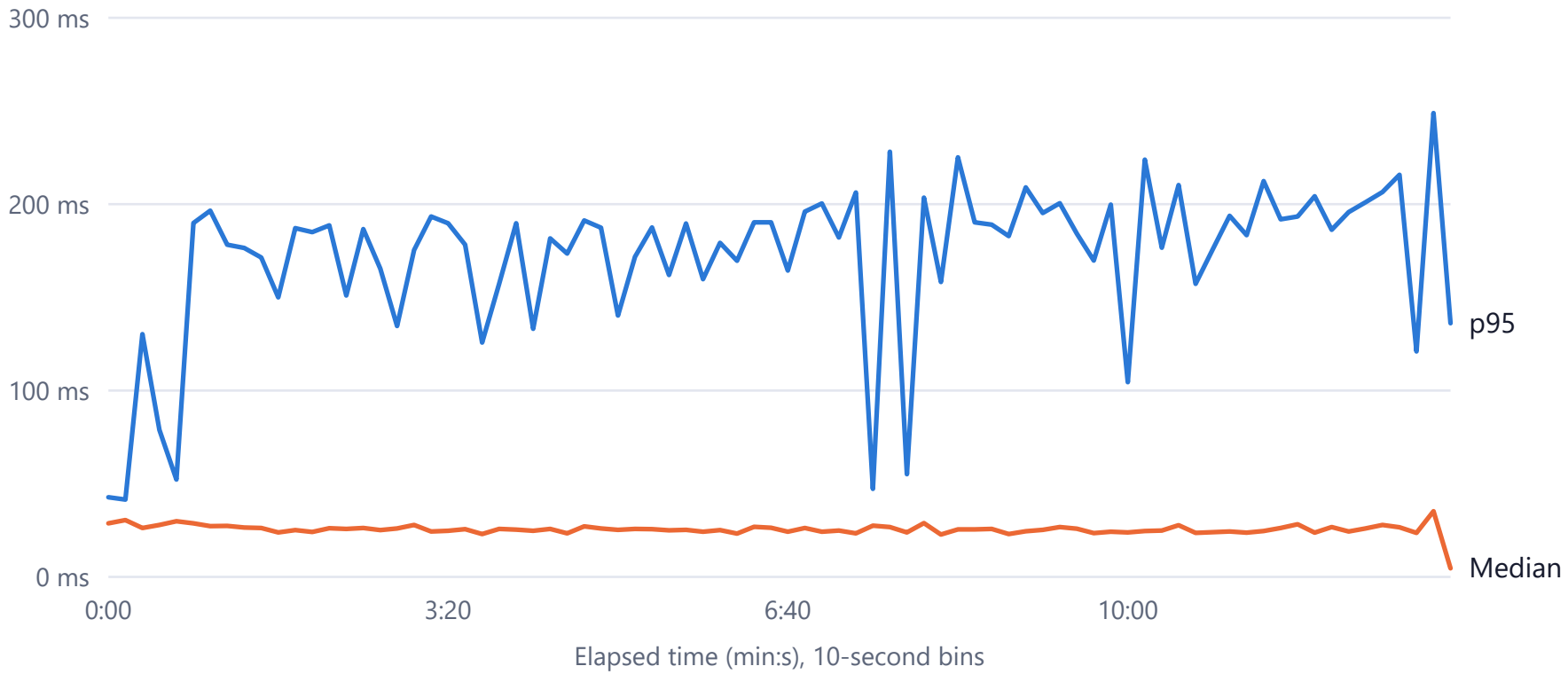
Test 1. Load: 50 users for 10 minutes

Ramp to 50 users over 2 minutes, hold for 10 minutes, ramp down over 1 minute. Figures cover the 10-minute hold. Throughput 28.6 requests/s, 0 errors in 17,182 requests.

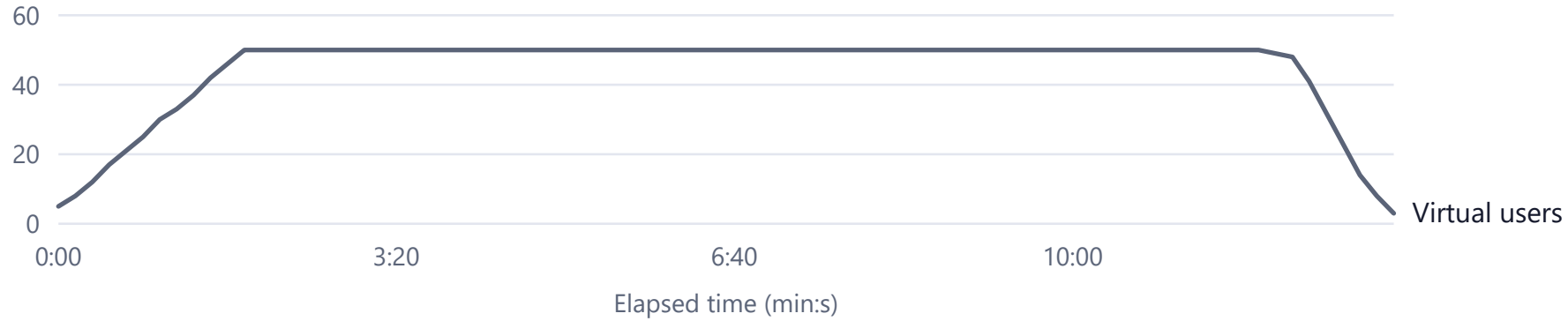
Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Login page GET index.php	1,298	18 ms	20 ms	29 ms	**31 ms**	90 ms	2.16	0	≤ 0.1 s
Log in (AJAX) POST ajax_login.php loginUser	1,298	27 ms	23 ms	38 ms	**40 ms**	92 ms	2.16	0	≤ 0.1 s
Requester home GET requester_home.php	981	28 ms	30 ms	40 ms	**43 ms**	80 ms	1.64	0	≤ 0.1 s
Home: store date POST store_date.php	981	13 ms	13 ms	15 ms	**16 ms**	19 ms	1.64	0	≤ 0.1 s
Home: day snapshot POST ajax_requester.php get_day_snapshot	981	20 ms	22 ms	29 ms	**30 ms**	61 ms	1.64	0	≤ 0.1 s
Request form GET forms.php	973	29 ms	31 ms	40 ms	**43 ms**	86 ms	1.62	0	≤ 0.1 s
Form: disabled dates POST ajax_forms.php get_disabled_dates	973	25 ms	24 ms	38 ms	**40 ms**	76 ms	1.62	0	≤ 0.1 s
Form: subjects POST ajax_forms.php get_subjects_by_grade	973	26 ms	30 ms	35 ms	**38 ms**	100 ms	1.62	0	≤ 0.1 s
Form: sections GET ajax_forms.php get_sections	973	26 ms	30 ms	35 ms	**37 ms**	96 ms	1.62	0	≤ 0.1 s
Conflict check POST ajax_forms.php check_conflict	2,269	26 ms	29 ms	37 ms	**40 ms**	52 ms	3.78	0	≤ 0.1 s
Submit request POST ajax_supervisor_action.php request_submission	976	177 ms	186 ms	234 ms	**244 ms**	278 ms	1.63	0	≤ 1 s
Request tracker GET requests.php	979	24 ms	26 ms	35 ms	**38 ms**	78 ms	1.63	0	≤ 0.1 s
Log out GET logout.php	1,298	2 ms	2 ms	4 ms	**5 ms**	57 ms	2.16	0	≤ 0.1 s
Admin home GET admin_home.php	319	34 ms	35 ms	46 ms	**50 ms**	68 ms	0.53	0	≤ 0.1 s
Admin: lab availability POST ajax_admin.php get_availability	319	20 ms	18 ms	31 ms	**33 ms**	40 ms	0.53	0	≤ 0.1 s

Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Admin: notifications GET ajax_admin.php notification_feed	319	24 ms	27 ms	35 ms	**37 ms**	41 ms	0.53	0	≤ 0.1 s
Admin: pending count GET ajax_admin.php get_pending_count	318	22 ms	28 ms	33 ms	**34 ms**	104 ms	0.53	0	≤ 0.1 s
Admin approvals GET admin_approve.php	319	219 ms	211 ms	356 ms	**375 ms**	457 ms	0.53	0	≤ 1 s
Approve request POST ajax_admin_action.php force_approve	318	200 ms	211 ms	254 ms	**264 ms**	321 ms	0.53	0	≤ 1 s
Inventory GET inventory.php	317	22 ms	24 ms	34 ms	**36 ms**	43 ms	0.53	0	≤ 0.1 s
All requests	**17,182**	**38 ms**	**25 ms**	**47 ms**	**187 ms**	**457 ms**	**28.64**	**0**	≤ 1 s





Active virtual users over the same time axis:



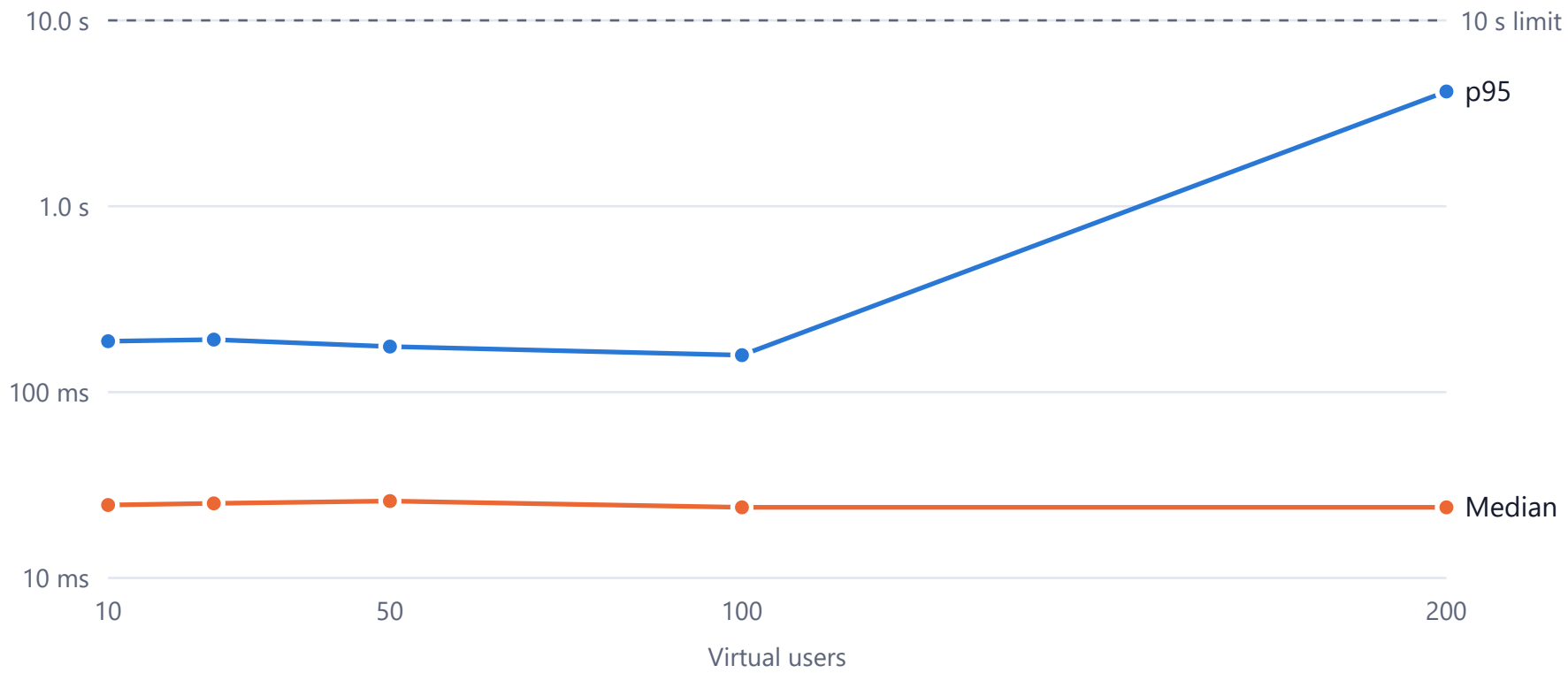
Run	Apache CPU (mean / max)	Apache memory	MariaDB CPU	MariaDB memory	k6 CPU	Whole machine CPU
Load, 50 users	2.3% / 4.5%	172 / 175 MB	0.6% / 2.2%	62 / 63 MB	1.0% / 3.9%	12.6% / 22.4%

Test 2. Stress: 10 to 200 users

Steps of 10, 25, 50, 100 and 200 users, each a 20 s ramp, a 3-minute hold and a 10 s ramp down on a freshly restored baseline. Figures cover the 3-minute hold. Stop rule: p95 above 10 s or error rate above 5%.

Users	Req/s	Average	Median	p90	p95	Max	Errors	Apache CPU	MariaDB CPU	Machine CPU	Limits
10	5.8	36 ms	25 ms	44 ms	**188 ms**	294 ms	0.00%	0.4%	0.1%	7.0%	within
25	14.4	37 ms	25 ms	45 ms	**192 ms**	287 ms	0.00%	1.1%	0.2%	9.0%	within
50	28.6	36 ms	26 ms	45 ms	**176 ms**	289 ms	0.00%	2.1%	0.4%	11.1%	within
100	57.6	34 ms	24 ms	44 ms	**158 ms**	275 ms	0.00%	4.3%	0.9%	17.0%	within
200	94.2	404 ms	24 ms	202 ms	**4.14 s**	6.65 s	0.00%	10.8%	2.5%	33.5%	within
300	–	–	–	–	–	–	–	–	–	–	not completed





Logarithmic vertical axis. The dashed line is the 10 s stop limit (Nielsen's limit for keeping attention).

Capacity: ≥ 200 users. Last completed step within limits. The 300-VU step was stopped after ~3 min when Claude Code reaped the background task for low system memory; the upper bound was not measured. Capacity reported as >= 200 VUs by decision of the researcher.

Where the time goes at 200 users

Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Login page GET index.php	1,274	35 ms	14 ms	29 ms	**31 ms**	2.06 s	7.08	0	≤ 0.1 s

Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Log in (AJAX) POST ajax_login.php loginUser	1,273	29 ms	24 ms	44 ms	**54 ms**	119 ms	7.07	0	≤ 0.1 s
Requester home GET requester_home.php	948	33 ms	30 ms	52 ms	**67 ms**	152 ms	5.27	0	≤ 0.1 s
Home: store date POST store_date.php	948	14 ms	12 ms	21 ms	**28 ms**	62 ms	5.27	0	≤ 0.1 s
Home: day snapshot POST ajax_requester.php get_day_snapshot	948	19 ms	18 ms	30 ms	**32 ms**	71 ms	5.27	0	≤ 0.1 s
Request form GET forms.php	959	1.35 s	37 ms	4.44 s	**4.85 s**	6.27 s	5.33	0	≤ 10 s
Form: disabled dates POST ajax_forms.php get_disabled_dates	959	27 ms	24 ms	40 ms	**49 ms**	92 ms	5.33	0	≤ 0.1 s
Form: subjects POST ajax_forms.php get_subjects_by_grade	959	26 ms	26 ms	40 ms	**49 ms**	101 ms	5.33	0	≤ 0.1 s
Form: sections GET ajax_forms.php get_sections	958	26 ms	25 ms	40 ms	**47 ms**	90 ms	5.32	0	≤ 0.1 s
Conflict check POST ajax_forms.php check_conflict	2,250	1.78 s	36 ms	4.64 s	**4.95 s**	6.53 s	12.50	0	≤ 10 s
Submit request POST ajax_supervisor_action.php request_submission	960	133 ms	133 ms	186 ms	**203 ms**	296 ms	5.33	0	≤ 1 s
Request tracker GET requests.php	957	22 ms	22 ms	35 ms	**39 ms**	78 ms	5.32	0	≤ 0.1 s
Log out GET logout.php	1,260	3 ms	2 ms	5 ms	**7 ms**	20 ms	7.00	0	≤ 0.1 s
Admin home GET admin_home.php	330	40 ms	38 ms	59 ms	**73 ms**	119 ms	1.83	0	≤ 0.1 s
Admin: lab availability POST ajax_admin.php get_availability	330	22 ms	18 ms	34 ms	**39 ms**	98 ms	1.83	0	≤ 0.1 s

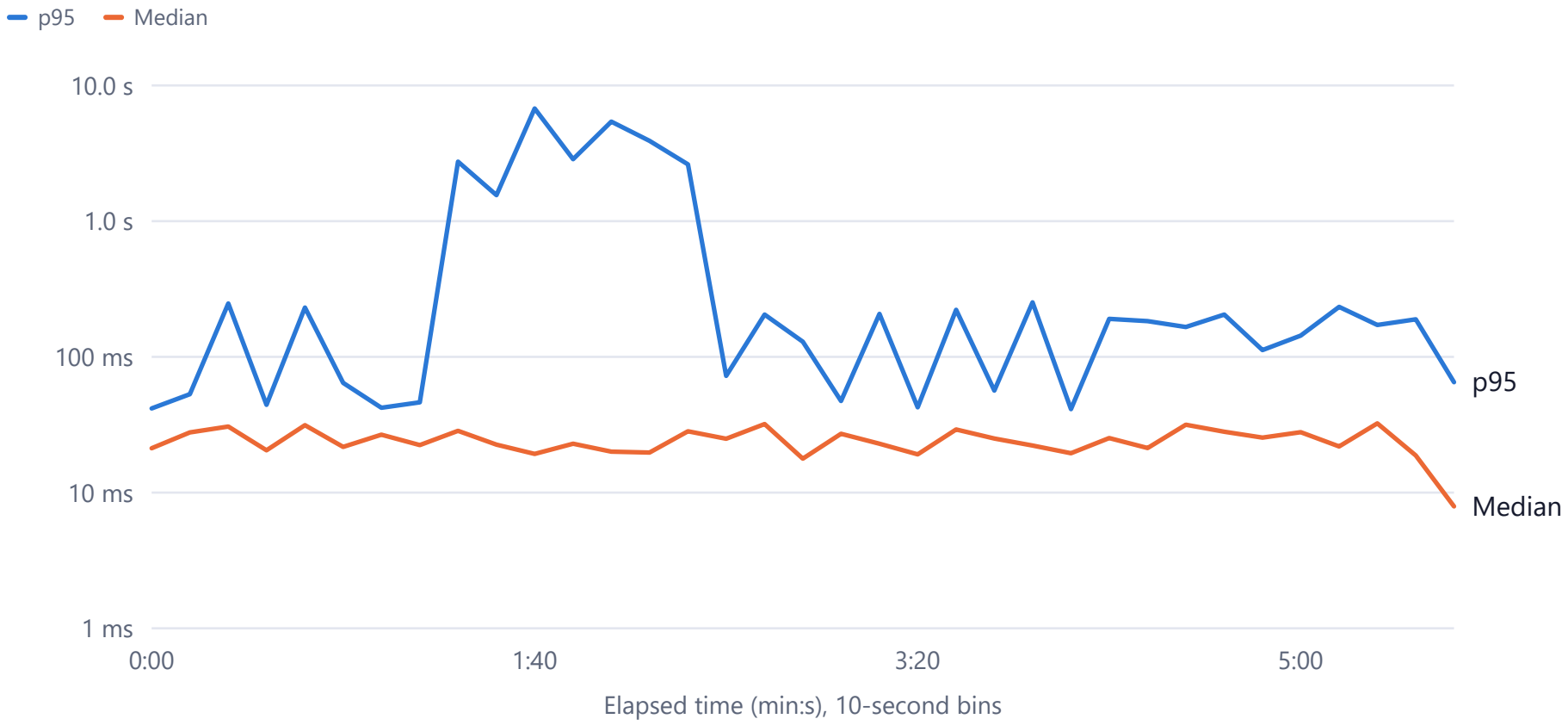
Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Admin: notifications GET ajax_admin.php notification_feed	330	21 ms	19 ms	34 ms	**37 ms**	61 ms	1.83	0	≤ 0.1 s
Admin: pending count GET ajax_admin.php get_pending_count	330	17 ms	14 ms	31 ms	**33 ms**	51 ms	1.83	0	≤ 0.1 s
Admin approvals GET admin_approve.php	334	1.87 s	578 ms	5.00 s	**5.39 s**	6.49 s	1.86	0	≤ 10 s
Approve request POST ajax_admin_action.php force_approve	326	1.58 s	171 ms	4.59 s	**4.95 s**	6.65 s	1.81	0	≤ 10 s
Inventory GET inventory.php	320	21 ms	21 ms	35 ms	**38 ms**	53 ms	1.78	0	≤ 0.1 s
All requests	**16,953**	**404 ms**	**24 ms**	**202 ms**	**4.14 s**	**6.65 s**	**94.18**	**0**	≤ 10 s

The slow requests were spread evenly over the 3-minute hold (about 85 per 10 seconds) and concentrated in four endpoints: the conflict check, the request form, admin approvals and approve request. These are the requests that follow a 3–10 s think time. The delay was in time-to-first-byte (p95 4.0 s), while the TCP connection was accepted within 13 ms, and the slow responses clustered between 3 and 5 s. Machine CPU averaged 34% and at least 4 GB of memory stayed free. This matches Apache's Windows thread model: 150 worker threads, each held by an idle keep-alive connection for up to 5 s. With 200 users, a request that arrives on a new connection after a pause longer than the keep-alive timeout waits until another connection times out. At 100 users (fewer connections than threads) there is no such tail. This explanation fits all observations but was not confirmed by changing the Apache configuration, which was outside the scope of this test.

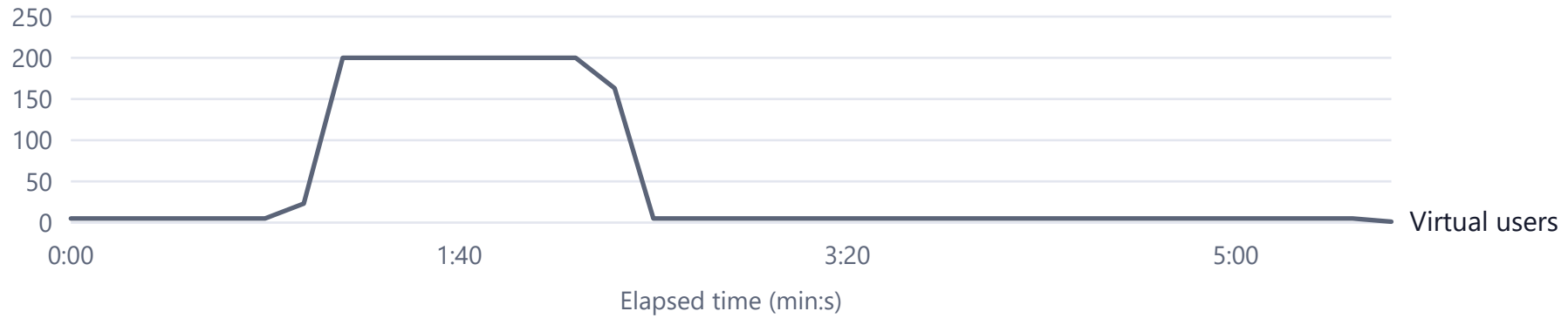
Test 3. Spike: 5 → 200 → 5 users

One minute at 5 users, jump to 200 users in 10 s, hold 1 minute, drop back to 5 users in 10 s, then 3 minutes at 5 users.

Phase	Requests	Req/s	Median	p90	p95	Max	Errors
Baseline, 5 users (1 min)	233	3.9	27 ms	42 ms	**51 ms**	305 ms	0
Jump to 200 (10 s)	1,009	100.9	22 ms	40 ms	**46 ms**	3.06 s	0
Peak, 200 users (1 min)	5,765	96.1	22 ms	193 ms	**3.11 s**	9.58 s	0
Drop to 5 (10 s)	315	31.5	28 ms	735 ms	**2.62 s**	3.47 s	0
Recovery, 5 users (3 min)	521	2.9	25 ms	46 ms	**207 ms**	278 ms	0



Active virtual users over the same time axis:



Logarithmic vertical axis.

No request failed during the spike. At the peak the median stayed at 22 ms and the p95 rose to 3.11 s (maximum 9.58 s), the same pattern as the 200-user stress step. **Recovery was immediate:** in the first 10 s after the users were back at 5, the p95 was 72 ms, and the recovery-phase median (25 ms) matched the baseline (27 ms).

Recovery rule. The planned rule (10-s p95 back within 1.2 × the baseline p95 for 30 s) is not met at any point, including during the baseline itself: at 5 users a 10-s bin holds about 30 requests, and its p95 swings between 42 and 247 ms depending on whether a ~200 ms submit or approve call falls in it, so the 1-minute baseline p95 (51 ms) is not a stable reference. Against the steady low-load p95 of the 10-user stress step (188 ms, about 1,000 requests), chosen after seeing the data, the p95 was back within limits 10 s after the drop began.

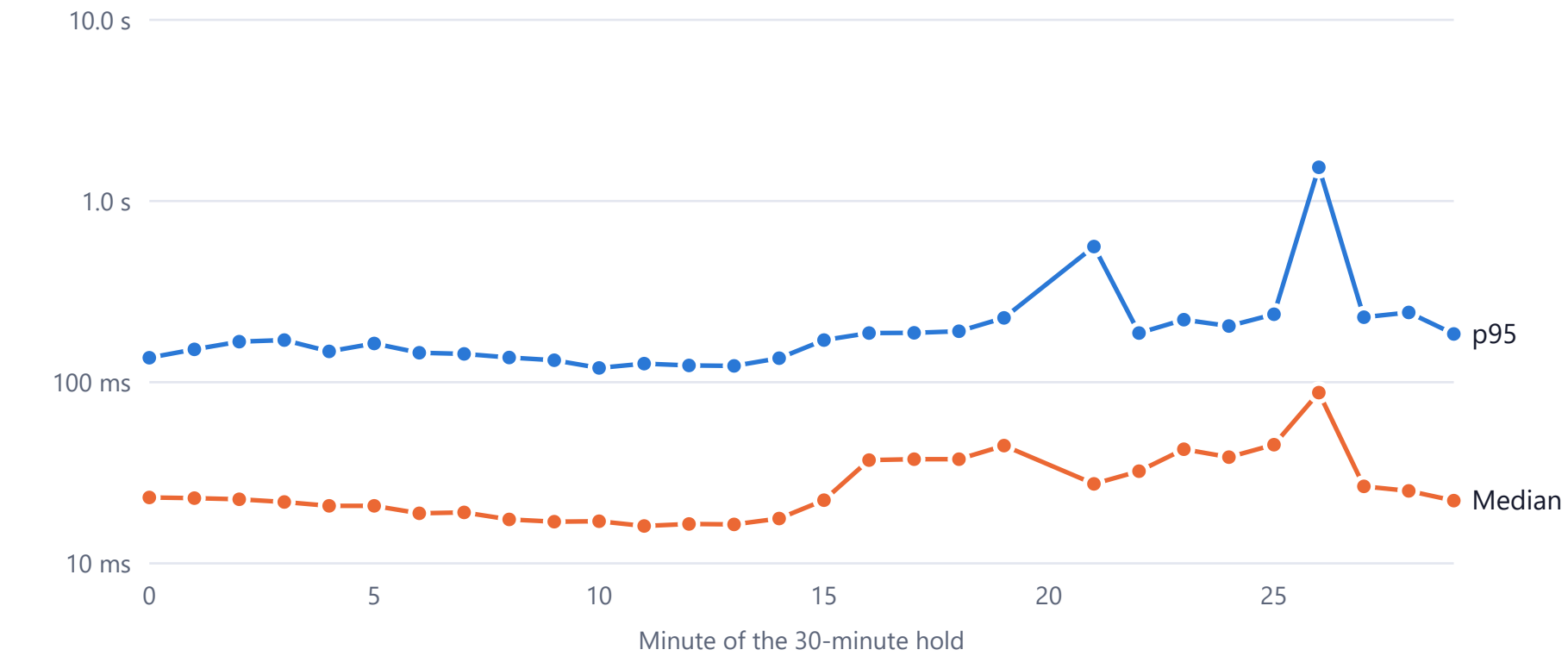
Test 4. Soak: 100 users for 30 minutes

50% of the measured capacity for 30 minutes (after a 1-minute ramp). 92,763 requests at 51.5 requests/s; 13 errors (0.014%: 4 timeout, 9 connection).

Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Login page GET index.php	6,955	23 ms	15 ms	30 ms	**38 ms**	2.07 s	3.86	0	≤ 0.1 s
Log in (AJAX) POST ajax_login.php loginUser	6,955	42 ms	29 ms	57 ms	**83 ms**	2.38 s	3.86	0	≤ 0.1 s
Requester home GET requester_home.php	5,405	75 ms	32 ms	75 ms	**119 ms**	126.59 s	3.00	0	≤ 1 s
Home: store date POST store_date.php	5,405	19 ms	13 ms	24 ms	**35 ms**	1.56 s	3.00	0	≤ 0.1 s
Home: day snapshot POST ajax_requester.php get_day_snapshot	5,405	27 ms	22 ms	36 ms	**48 ms**	1.43 s	3.00	0	≤ 0.1 s
Request form GET forms.php	5,411	53 ms	33 ms	62 ms	**94 ms**	3.51 s	3.01	0	≤ 0.1 s
Form: disabled dates POST ajax_forms.php get_disabled_dates	5,411	44 ms	30 ms	64 ms	**102 ms**	2.61 s	3.01	0	≤ 1 s
Form: subjects POST ajax_forms.php get_subjects_by_grade	5,411	37 ms	29 ms	55 ms	**89 ms**	2.41 s	3.01	0	≤ 0.1 s
Form: sections GET ajax_forms.php get_sections	5,411	36 ms	29 ms	53 ms	**84 ms**	1.86 s	3.01	0	≤ 0.1 s
Conflict check POST ajax_forms.php check_conflict	12,376	39 ms	28 ms	55 ms	**84 ms**	2.89 s	6.88	6	≤ 0.1 s
Submit request POST ajax_supervisor_action.php request_submission	5,416	174 ms	156 ms	249 ms	**322 ms**	3.14 s	3.01	3	≤ 1 s
Request tracker GET requests.php	5,413	47 ms	30 ms	64 ms	**88 ms**	2.14 s	3.01	0	≤ 0.1 s
Log out GET logout.php	6,955	8 ms	2 ms	7 ms	**10 ms**	1.70 s	3.86	0	≤ 0.1 s
Admin home GET admin_home.php	1,547	141 ms	43 ms	95 ms	**115 ms**	126.65 s	0.86	0	≤ 1 s

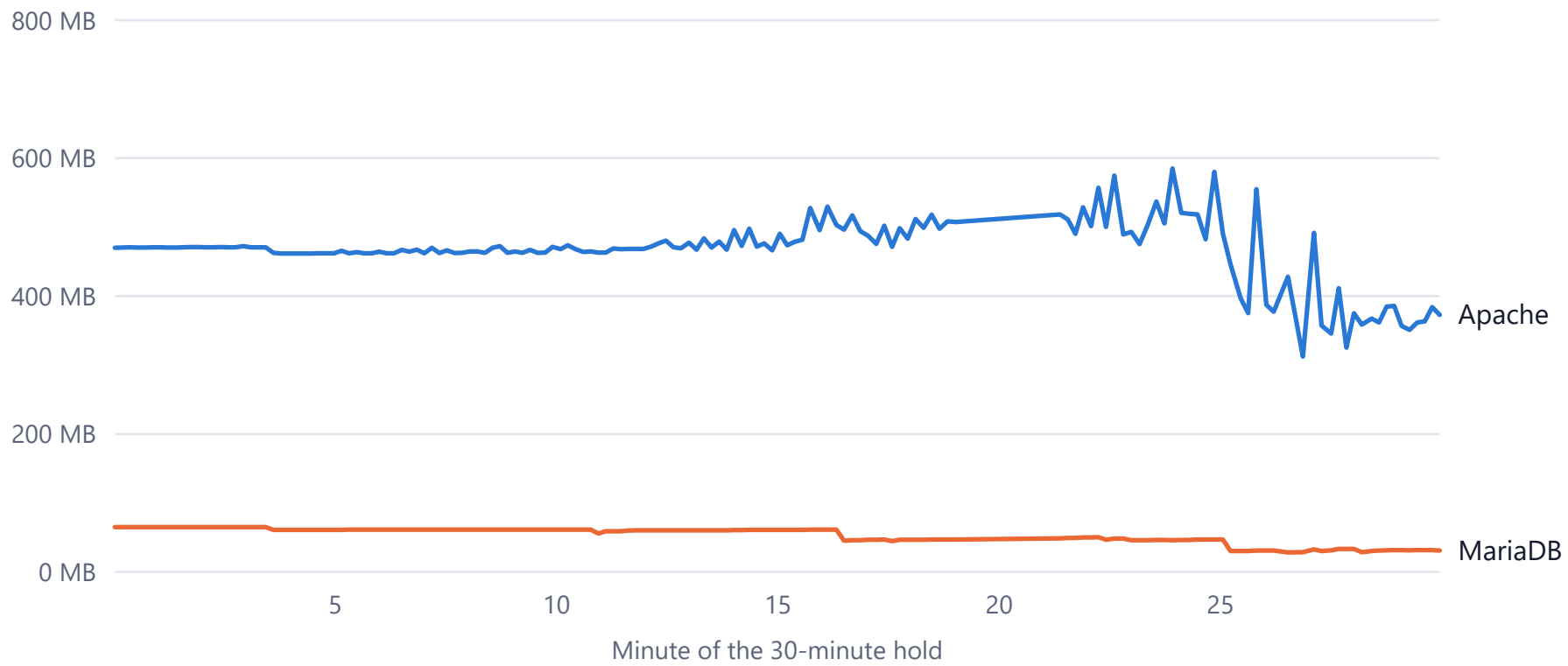
Endpoint	Requests	Average	Median	p90	p95	Max	Req/s	Errors	Nielsen band (p95)
Admin: lab availability POST ajax_admin.php get_availability	1,547	25 ms	22 ms	35 ms	**45 ms**	641 ms	0.86	0	≤ 0.1 s
Admin: notifications GET ajax_admin.php notification_feed	1,547	27 ms	25 ms	40 ms	**49 ms**	479 ms	0.86	0	≤ 0.1 s
Admin: pending count GET ajax_admin.php get_pending_count	1,547	22 ms	20 ms	34 ms	**41 ms**	320 ms	0.86	0	≤ 0.1 s
Admin approvals GET admin_approve.php	1,547	2.39 s	959 ms	4.41 s	**6.09 s**	130.83 s	0.86	4	≤ 10 s
Approve request POST ajax_admin_action.php force_approve	1,550	200 ms	167 ms	243 ms	**289 ms**	3.23 s	0.86	0	≤ 1 s
Inventory GET inventory.php	1,549	24 ms	22 ms	37 ms	**48 ms**	1.07 s	0.86	0	≤ 0.1 s
All requests	**92,763**	**88 ms**	**26 ms**	**112 ms**	**192 ms**	**130.83 s**	**51.53**	**13**	≤ 1 s





Per-minute median and p95 of all requests, logarithmic axis.





Private working set, sampled every 2 s (every fifth sample plotted).

Measure	Linear trend	R^2	p	First value	Last value
p95 per minute	+11.25 ms/min	0.14	.042	136.5	184.5
Median per minute	+0.89 ms/min	0.29	.0028	23.1	22.2
Apache memory	-1.34 MB/min	0.07	< .001	470	369.2
MariaDB memory	-1.19 MB/min	0.82	< .001	64.9	30.9
Apache CPU	+0.43 points/min	0.41	< .001	4.9	11.3

Measure	Linear trend	R^2	p	First value	Last value
MariaDB CPU	+0.30 points/min	0.55	< .001	0.7	8.5

No memory growth while the machine was undisturbed. In minutes 0–15 Apache's private memory averaged 468 MB with a trend of +0.29 MB per minute (R^2 = 0.05, about 4 MB in 15 minutes), MariaDB's averaged 62 MB with a slight downward trend, and the per-minute p95 of all requests did not rise (−2.6 ms per minute). After minute 15 both processes' memory was shaped by the host (see below): it fluctuated, then fell to about 390 MB and 31 MB as Windows trimmed working sets under memory pressure, so the full-run memory trends in the table are not evidence either way. **One page did slow down steadily**, and the cause is data growth rather than a leak. Every soak session submits a request, so about 5,400 requests accumulated and about 4,000 were still pending at the end. The admin approvals page lists every pending request (with its full data repeated in three buttons per row) without paging, and its median rose from 166 ms in minutes 0–5 to 499 ms, 890 ms, 2.9 s, 3.9 s and 4.9 s in the following 5-minute blocks. Over the same time Apache CPU rose from 4.9% to 11.3% and MariaDB CPU from 0.7% to 8.5% of the machine at a constant number of users.

Host interference. From about minute 15, other programmes on the laptop competed for the machine: mean machine CPU rose from 21% (minutes 0–15) to 53%, while Apache, MariaDB and k6 together used about 32%, and available memory fell below 1 GB at minute 23 and as low as 22 MB. At about minute 21, with the machine's CPU at 92–97%, every request in flight stalled for about 2 minutes (maximum 130.8 s; 4 timeouts and 9 connection errors). The p95 outliers at minutes 20 and 25, the rise of the overall median after minute 15 and the errors coincide with these episodes. They are a property of the test machine, not of Reserve-a-Lab, and they produce the full-run p95 trend in the table. The admin-page growth started before them.

Run	Apache CPU (mean / max)	Apache memory	MariaDB CPU	MariaDB memory	k6 CPU	Whole machine CPU
Soak, 100 users	9.5% / 34.8%	465 / 591 MB	4.3% / 16.6%	53 / 65 MB	6.2% / 34.5%	35.0% / 99.9%
Spike, 5→200	1.7% / 10.5%	469 / 470 MB	0.3% / 2.3%	63 / 63 MB	0.6% / 5.2%	10.2% / 31.5%
Stress, 10 users	0.4% / 0.9%	173 / 173 MB	0.1% / 0.2%	63 / 63 MB	0.2% / 0.6%	7.0% / 14.6%
Stress, 25 users	1.1% / 2.4%	173 / 173 MB	0.2% / 0.5%	64 / 64 MB	0.3% / 1.0%	9.0% / 16.2%
Stress, 50 users	2.1% / 3.5%	173 / 173 MB	0.4% / 0.8%	60 / 62 MB	0.8% / 1.8%	11.1% / 21.7%
Stress, 100 users	4.3% / 6.7%	315 / 318 MB	0.9% / 2.0%	65 / 65 MB	1.8% / 4.3%	17.0% / 24.9%

Run	Apache CPU (mean / max)	Apache memory	MariaDB CPU	MariaDB memory	k6 CPU	Whole machine CPU
Stress, 200 users	10.8% / 29.4%	467 / 473 MB	2.5% / 13.4%	64 / 64 MB	5.0% / 13.9%	33.5% / 88.3%

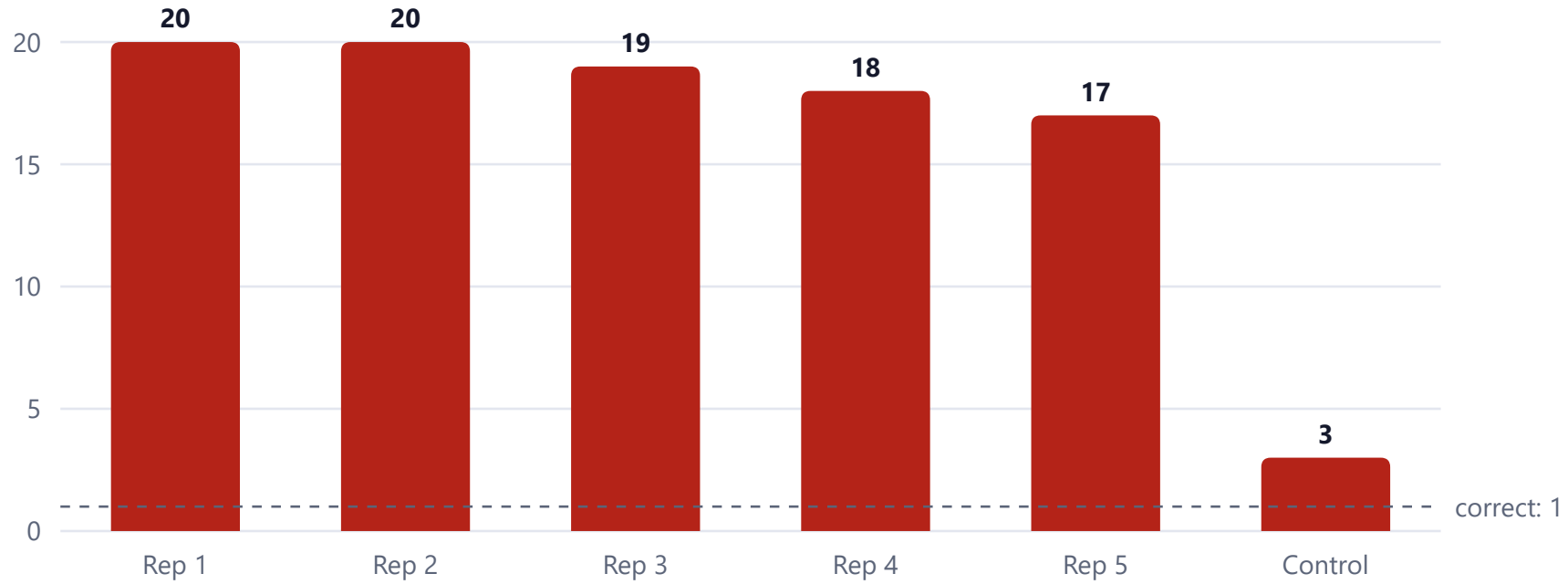
Test 5. Concurrency and double booking

Each repetition starts from the restored baseline. All virtual users log in and load their pages first, then wait for the same wall-clock instant (a barrier set in k6 `setup()`) before sending the critical request. The largest gap between the first and last request at a barrier was 4 ms. After each run the database was queried directly; the query output is reproduced below as evidence. These are the results **before** the fixes; the re-runs after them are in After fixes.

5a. Same lab, date and time slot

FAIL. 20 teachers requested Science Laboratory 3, 9:00–11:00, on the same date at the same instant, and 20 admin sessions then approved them at the same instant. The number of approved requests for that one slot was 20, 20, 19, 18, 17 in the five repetitions. A correct system approves at most 1.

Run	Date	Requests for the slot	Approved	Still pending	Result
Repetition 1	2027-03-15	20	**20**	0	FAIL
Repetition 2	2027-03-16	20	**20**	0	FAIL
Repetition 3	2027-03-17	20	**19**	1	FAIL
Repetition 4	2027-03-18	20	**18**	2	FAIL
Repetition 5	2027-03-19	20	**17**	3	FAIL
Control: one by one, 3 s apart	2027-03-15	3	**3**	0	FAIL



Approved requests for the same slot per run; the dashed line is the correct maximum.

Why. The approve action (`ajax/ajax_admin_action.php` , `force_approve` , line 117; the same applies to the multi-stage path in `ajax_supervisor_action.php`) updates the request to Approved without checking the slot on the server, without a transaction or row lock, and without a unique constraint in the database. The only check is a separate `check_conflict` call that the browser makes before showing the approve dialog. The race runs gave every admin the most favourable client: each ran that check correctly and skipped the approval if it reported an approved conflict. Even so, 17–20 approvals went through, because all checks ran before any approval was written (a check-then-act race). Only the few admins whose check arrived after another approval had been written were stopped (0, 0, 1, 2 and 3 per repetition).

It also fails without concurrency. The approvals page sends `check_conflict` without the `csrf_token` field the endpoint requires (`admin_approve.php` line 1604 against `ajax_forms.php` line 446), so the browser receives HTTP 403 and never shows the "Severe Conflict" warning. In the control run, three admins approved three requests for the same slot 3 seconds apart through that real page path; the page's check returned 403 each time, a correctly sent

check would have reported "approved" for the second and third, and the server approved all three. The live warning on the request form (`forms.php` line 1327) has the same missing field. The submission endpoint actually used by the form (`ajax_supervisor_action.php`) has no slot check either.

5b. Last units of one inventory item

> **FAIL.** An item with 5 units left; 20 teachers each requested 2 units at the same instant and 20 admins approved all 20 requests at the same instant. In every repetition all 20 were approved, allocating 40 units against 5 in stock. The stock went below zero (−1) in 2 of 5 repetitions.

Run	Starting stock	Requests	Approved	Units allocated	Stock after	Over-allocated	Below zero	Result
Repetition 1	5	20	**20**	40	**0**	yes	no	**FAIL**
Repetition 2	5	20	**20**	40	**-1**	yes	yes	**FAIL**
Repetition 3	5	20	**20**	40	**0**	yes	no	**FAIL**
Repetition 4	5	20	**20**	40	**0**	yes	no	**FAIL**
Repetition 5	5	20	**20**	40	**-1**	yes	yes	**FAIL**
Control: one by one, 3 s apart	5	3	**3**	6	**0**	yes	no	**FAIL**

Why. Stock is checked at submission against the current quantity, but nothing is reserved, so every request for 2 of 5 units passes. At approval, `scilab_deduct_inventory()` (`helperFiles/variableDeclarations.php`) approves regardless of stock and deducts `min(requested, quantity read)` with a separate `UPDATE … SET quantity = quantity - ? WHERE quantity > 0` . When two approvals read the same quantity (for example 1) before either writes, the second subtracts from a value that is no longer there, and the signed `INT` column accepts −1. The control run shows that over-allocation happens even one approval at a time (3 approved, 6 units against 5); going below zero needs concurrency.

5c. Control: the inventory page's manual deduction

PASS, 5 of 5. 20 admin sessions each deducted 1 unit from an item with 5 units at the same instant, using the inventory page's deduction (`ajax_inventory.php` , which uses a transaction, `SELECT … FOR UPDATE` and `quantity >= ?`). Every repetition ended at 0 with exactly 5 deductions logged (5→4→3→2→1→0) and 15 refusals ("only 0 in stock"). This shows that the test detects correct locking as well as missing locking.

Run	Starting stock	Requests	Deductions logged	Units deducted	Stock after	Result
Repetition 1	5	20	5	5	0	PASS
Repetition 2	5	20	5	5	0	PASS
Repetition 3	5	20	5	5	0	PASS
Repetition 4	5	20	5	5	0	PASS
Repetition 5	5	20	5	5	0	PASS

Database evidence

Raw output of the queries run after each repetition (also saved in `loadtests/results/*_db.txt`).

▶ conc_inventory_race_r1

▶ conc_inventory_race_r2

▶ conc_inventory_race_r3

▶ conc_inventory_race_r4

▶ conc_inventory_race_r5

▶ conc_inventory_sequential_r1

▶ conc_manual_race_r1

▶ conc_manual_race_r2

▶ conc_manual_race_r3

▶ conc_manual_race_r4

▶ conc_manual_race_r5

▶ conc_slot_race_r1

▶ conc_slot_race_r2

▶ conc_slot_race_r3

▶ conc_slot_race_r4

▶ conc_slot_race_r5

▶ conc_slot_sequential_r1

After fixes

The three concurrency failures above were fixed on the server side (branch `fix/concurrency`) and tests 5a, 5b and 5c were re-run with the same scripts, the same 20 simultaneous teachers and admins, the same baseline restored before every repetition and the same database queries. The **Before** columns below are the original runs, unchanged and still shown in full above; the **After** columns are the re-runs. The database was backed up before the re-runs and restored afterwards; outbound SMTP was blocked again for the duration.

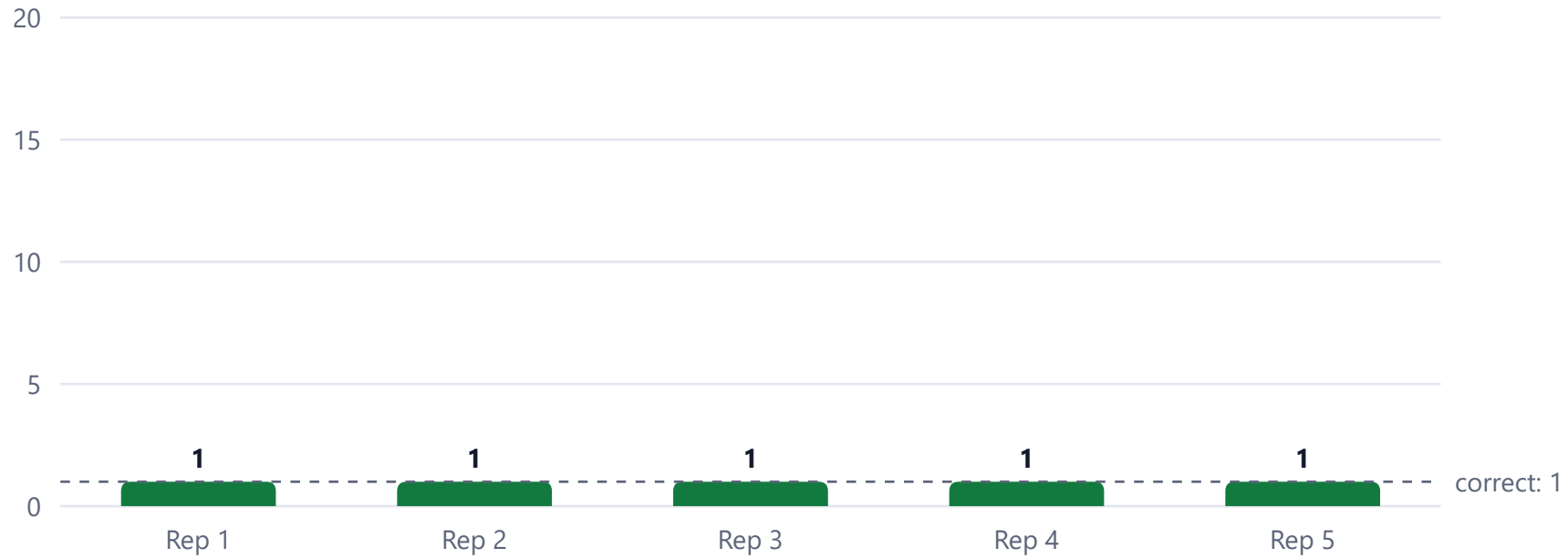
What changed

- **Approval is one transaction** (`scilab_approve_request()` in `helperFiles/variableDeclarations.php`), used by `force_approve` and `approve` in `ajax/ajax_admin_action.php` and by the final stage of the multi-stage path in `ajax/ajax_supervisor_action.php`. It locks the lab's `scilab_availability` row (`SELECT … FOR UPDATE`), refuses if an approved request for the same lab and date overlaps, then updates the status and deducts stock. A refusal rolls everything back.
- **Stock**: `scilab_deduct_inventory()` locks the item rows (`SELECT … FOR UPDATE`, in id order), refuses the approval with a message if any item has too little stock, and deducts with `WHERE quantity >= ?`, the same pattern as the inventory page's manual deduction. No schema change.
- **Submission** (`request_submission`) refuses a slot that an approved request already holds.
- **Conflict warning**: `admin_approve.php` and `forms.php` now send the `csrf_token` field with `check_conflict`, so the "Severe Conflict" warning can show again.

5a after the fix: same lab, date and time slot

> **PASS.** Approved requests for the one slot in the five repetitions: 1, 1, 1, 1, 1 (before: 20, 20, 19, 18, 17). The correct number is at most 1.

Run	Date	Approved, before	Approved, after	Stopped by their own check	Refused by the server	Still pending	Result
Repetition 1	2027-03-15	20	**1**	0	19	19	**PASS**
Repetition 2	2027-03-16	20	**1**	0	19	19	**PASS**
Repetition 3	2027-03-17	19	**1**	0	19	19	**PASS**
Repetition 4	2027-03-18	18	**1**	0	19	19	**PASS**
Repetition 5	2027-03-19	17	**1**	2	17	19	**PASS**
Control: one by one, 3 s apart	2027-03-15	3 of 3	**1 of 3**	–	2	2	**PASS**



Drawn on the same 0–20 axis as the Before chart in 5a. The dashed line is the correct maximum.

The admins' correct conflict checks still race: in repetitions 1 to 4 all 20 checks reported only "pending" because they ran before any approval was written, and in repetition 5 two admins' checks happened to see the approval and skipped it (before the fix this stopped 0 to 3 admins per repetition). What changed is that the server now decides: the first approval to take the lab lock was approved and every other admin who sent an approval was refused with "Cannot approve: Science Laboratory 3 is already booked …" (19 in repetitions 1 to 4 and 17 in repetition 5, where two had already stopped themselves). In the one-by-one control the second and third approvals were refused as well, so the sequential failure is fixed too.

5b after the fix: last units of one inventory item

PASS. Starting from 5 units and 20 simultaneous approvals of 2 units each, 2, 2, 2, 2, 2 requests were approved per repetition (4, 4, 4, 4, 4 units allocated) and the stock ended at 1, 1, 1, 1, 1. Before: 20 approved, 40 units allocated, stock 0 or −1.

Run	Before: approved / units allocated / stock after	After: approved / units allocated / stock after	Refused by the server	Over-allocated	Below zero	Result
Repetition 1	20 / 40 / 0	**2** / 4 / **1**	18	no	no	**PASS**
Repetition 2	20 / 40 / -1	**2** / 4 / **1**	18	no	no	**PASS**
Repetition 3	20 / 40 / 0	**2** / 4 / **1**	18	no	no	**PASS**
Repetition 4	20 / 40 / 0	**2** / 4 / **1**	18	no	no	**PASS**
Repetition 5	20 / 40 / -1	**2** / 4 / **1**	18	no	no	**PASS**
Control: one by one, 3 s apart	3 / 6 / 0	**2** / 4 / **1**	1	no	no	**PASS**

Two requests (4 units) fit in 5 units; the remaining 18 were refused with "Cannot approve: not enough stock for … (2 requested, only 1 … left). Nothing was deducted." and stayed pending. The stock never went below zero and no approval was written without its deduction. The one-by-one control, which used to allocate 6 units against 5, now approves two and refuses the third.

5c after the fix: manual deduction (control)

> **PASS, 5 of 5.** The inventory page's manual deduction was not changed; re-running it confirms that the new approval code did not disturb it and that the test setup is unchanged.

Run	Before: deductions / units / stock after	After: deductions / units / stock after	Result
Repetition 1	5 / 5 / 0	5 / 5 / 0	**PASS**
Repetition 2	5 / 5 / 0	5 / 5 / 0	**PASS**
Repetition 3	5 / 5 / 0	5 / 5 / 0	**PASS**
Repetition 4	5 / 5 / 0	5 / 5 / 0	**PASS**
Repetition 5	5 / 5 / 0	5 / 5 / 0	**PASS**

Did the locking slow things down? 50-user load test, before and after

The 10-minute, 50-user test (test 1) was repeated with the fixed code, on a freshly restored baseline and with the same scripts apart from the changes below. Approvals now take a per-lab lock, so a slowdown would show first in the approve and submit requests.

All requests, 10-minute hold	Before	After	Change
Requests	17,182	17,290	+1%

All requests, 10-minute hold	Before	After	Change
Throughput (requests/s)	28.6	28.8	+1%
Median	25 ms	25 ms	-2%
p90	47 ms	47 ms	-1%
p95	187 ms	176 ms	-6%
p99	256 ms	250 ms	-2%
Max	457 ms	456 ms	0%
Errors	0	0	–

Endpoint	Requests, before	Requests, after	Median, before	Median, after	p95, before	p95, after	p95 change	Errors, after
Conflict check	2269	2291	29 ms	30 ms	40 ms	41 ms	+2%	0
Submit request	976	981	186 ms	174 ms	244 ms	236 ms	-3%	0
Approve request	318	320	211 ms	192 ms	264 ms	256 ms	-3%	0
Admin approvals	319	319	211 ms	224 ms	375 ms	372 ms	-1%	0
Request form	973	985	31 ms	30 ms	43 ms	42 ms	-1%	0

Resource	Before	After
Apache CPU (mean)	2.3%	2.4%
MariaDB CPU (mean)	0.6%	0.6%

Resource	Before	After
Whole machine CPU (mean)	12.6%	13.7%

Outcomes over the whole after run, including ramp-up and ramp-down: approvals 370 approved and 1 refused by the server; submissions 1126 accepted and 24 refused because an approved request held the slot. A refusal is a correct answer, not an error, and was not counted as one. The k6 journeys now send `csrf_token` with both conflict checks, as the fixed pages do, so those calls return 200 instead of 403 and run the conflict query; the before run replayed the pages as they were.

Limits of the after-fix results

- Same laptop, same barrier technique, same 20 virtual users and 5 repetitions as before; a pass in five repetitions is evidence, not a proof. The lock and the transaction are what guarantee the result.
- Only the paths exercised by these tests were run: `force_approve` from the admin approvals page. The multi-stage path (`ajax_supervisor_action.php` , final stage) uses the same helper but was not part of the race tests.
- Not covered: two admins approving the *same* request at the same time (the request is not checked for being already approved), and `supervisor_approve.php` , whose conflict check still omits `csrf_token` .
- Behaviour change: approval is now refused when stock is short, where it used to approve and deduct what was left. For reagents, submission checks the total across rows but the deduction takes from one row, so a request spread across rows can pass submission and be refused at approval.
- The SMTP block was in place, so notification e-mail time is not included, as in the original runs.

Database evidence after the fixes

Raw output of the same queries after each re-run (also saved in `loadtests/results/*_after_*_db.txt`).

▶ **conc_inventory_race_after_r1**

▶ **conc_inventory_race_after_r2**

▶ conc_inventory_race_after_r3

▶ conc_inventory_race_after_r4

▶ conc_inventory_race_after_r5

▶ conc_inventory_sequential_after_r1

▶ conc_manual_race_after_r1

▶ conc_manual_race_after_r2

▶ conc_manual_race_after_r3

▶ conc_manual_race_after_r4

▶ conc_manual_race_after_r5

▶ conc_slot_race_after_r1

▶ conc_slot_race_after_r2

▶ conc_slot_race_after_r3

▶ conc_slot_race_after_r4

▶ conc_slot_race_after_r5

▶ conc_slot_sequential_after_r1

Statistics: Lighthouse before and after

Mobile preset, median of 3 runs per page, from the Lighthouse report. The login page is excluded because Lighthouse could not score it before the changes (NO_FCP), leaving n = 8 paired pages. Differences are after − before. Normality of the differences was checked with Shapiro–Wilk; the paired t-test is preferred when p ≥ .05, the Wilcoxon signed-rank test otherwise. Effect sizes: Cohen's d_z (mean difference / SD of differences) and the matched-pairs rank-biserial correlation r_{rb}. Holm-adjusted p-values correct for the four metrics.

Metric	Mean before	Mean after	Mean difference [95% CI]	Shapiro–Wilk W (p)	t(7)	p	p Holm	d_z	Wilcoxon W	p	p Holm	r_{rb}	Trust
Performance score (points)	60.9	86.0	25.1 [23.2, 27.0]	0.839 (.074)	30.96	< .001	< .001	10.95	0	.0078	.031	1.00	**paired t-test**
LCP (s)	8.67	3.67	-5.00 [-7.46, -2.54]	0.545 (< .001)	-4.81	.0019	.0039	-1.70	0	.0078	.031	-1.00	**Wilcoxon signed-rank**
FCP (s)	4.83	2.62	-2.22 [-2.47, -1.96]	0.902 (.304)	-20.82	< .001	< .001	-7.36	0	.0078	.031	-1.00	**paired t-test**
Page weight (KB)	1,743	497	-1,246 [-2,665, 173]	0.463 (< .001)	-2.08	.077	.077	-0.73	0	.0078	.031	-1.00	**Wilcoxon signed-rank**

- **Performance score and FCP:** the differences are consistent with normality, so the paired t-test applies; both are significant after Holm correction with very large effects.
- **LCP and page weight:** the differences are not normal (the About page lost 12.2 s of LCP and 5,442 KB, far more than any other page), so the Wilcoxon test is the one to trust. For page weight the two tests disagree: the t-test is not significant ($p = .077$) because the About outlier inflates the variance, while Wilcoxon finds every one of the 8 pages lighter ($p = .0078$).
- With 8 pairs and every page improving, $W = 0$ and $p = .0078$ is the smallest p-value the exact Wilcoxon test can give, so its p-value cannot grow with the size of the improvement; the effect size ($r_{rb} = 1.00$ for all metrics: every page improved) carries that information.
- Sensitivity: repeating the tests on the rounded values printed in the Lighthouse report gives the same conclusions (score t-test $p < .001$, FCP $p < .001$, LCP Wilcoxon $p = .0078$, page weight t-test $p = .076$ and Wilcoxon $p = .0078$).

Interpretation

Nielsen's response-time limits

Nielsen (1993) gives three limits: 0.1 s feels instantaneous, 1 s keeps the user's flow of thought, and 10 s is the limit for keeping attention on the task. The table counts endpoints by the band their p95 falls in.

Run	Endpoints with p95 ≤ 0.1 s	≤ 1 s	≤ 10 s	> 10 s	p95, all requests
Load, 50 users	17	3	0	0	187 ms
Stress, 10 users	18	2	0	0	188 ms
Stress, 25 users	18	2	0	0	192 ms
Stress, 50 users	17	3	0	0	176 ms
Stress, 100 users	17	3	0	0	158 ms
Stress, 200 users	15	1	4	0	4.14 s

Run	Endpoints with p95 ≤ 0.1 s	≤ 1 s	≤ 10 s	> 10 s	p95, all requests
Soak, 100 users	14	5	1	0	192 ms

At the expected peak of 50 users, 17 of 20 endpoints answer within 0.1 s at the 95th percentile and the rest within 1 s. The three that exceed 0.1 s are the actions that write and send e-mail (submit request, approve request) and the admin approvals page, which grows with the number of pending requests. No endpoint reaches the 10 s limit at any load up to 200 users; at 200 users four endpoints move into the 1–10 s band at the 95th percentile, where users notice the delay but stay on task. In the soak test only the admin approvals page left the 1 s band, because of data growth.

ISO/IEC 25010 performance efficiency

Sub-characteristic	Result	Evidence
Time behaviour	**Satisfactory** up to 100 users	Median ≈ 25 ms and p95 ≤ 192 ms from 10 to 100 users; at 200 users the median is unchanged but the p95 is 4.14 s. The admin approvals page degrades as pending requests accumulate (median 4.9 s with about 4,000 pending).
Resource utilization	**Satisfactory**	Apache used 2.3% of the machine's CPU at 50 users and 10.8% at 200; MariaDB below 3% up to 200 users. No memory growth in the undisturbed first 15 minutes of the soak (no sign of a leak). CPU per request rises as the request table grows.
Capacity	**≥ 200 users**	Within the limits at 200 concurrent users with 0% errors and 94 requests/s; above about 150 connections the Apache thread pool, not CPU, becomes the bottleneck. Upper limit not measured (host memory).

Functional correctness is a separate ISO/IEC 25010 characteristic (functional suitability), but the concurrency results belong in the same discussion: the system stays fast under concurrent use but does not keep its booking and stock rules under concurrent use, or in the slot case even under sequential use through the real page.

Limitations

- **Local laptop, not the campus server.** Server, database and load generator shared one laptop, which also ran other programmes. k6 used up to 14% of the machine's CPU at 200 users and competed with Apache. Memory pressure from other programmes stopped the 300-user step and disturbed the last half of the soak test. Absolute numbers on the campus server (different CPU, memory, operating system, possibly Linux with a different Apache MPM) will differ.
- **No network.** Requests never left the machine, so there was no network latency or bandwidth limit. Real users on the campus network or mobile data will see longer times; the Lighthouse report covers the front end on a throttled connection.
- **E-mail excluded from the timings.** With SMTP blocked, each notification attempt failed in milliseconds. In production every submission and approval sends its e-mails through Gmail inside the request (in the test database an approval attempted 36–56 notification e-mails and a submission 1–2), so real submit and approve times will be longer, possibly by seconds.
- **Simulated users.** Virtual users follow two scripted journeys with random think times; real users browse less regularly, use other pages and do not all submit a request every session. The 80/20 split and the think times are assumptions. Every session logged in afresh. Browser caching and static files were not part of the load.
- **Synthetic data.** 505 synthetic accounts were added (they lengthen the teacher list in the request form). Requests accumulated faster than in real use (one per requester session), which makes the data-growth effect on the admin approvals page appear within minutes; in real use it would build up over weeks if pending requests are not cleared.
- **Capacity is a lower bound.** The 300-user step did not finish, so capacity is reported as at least 200 concurrent users.
- **Lighthouse statistics.** n = 8 pages is small; the pages are not a random sample, and each value is a median of 3 runs. The About-page LCP value is the published median (17.7 s) because its saved report is a different run (14.6 s).
- **MariaDB counters.** `FLUSH STATUS` did not reset the server's status counters in this setup, so per-run MariaDB counters are not reported.

Reproducing

Scripts are in `loadtests/`; see `loadtests/README.md`. Raw k6 output (CSV), resource samples, PHP/Apache log extracts and summaries are in `loadtests/results/`.